

Comparative Analysis of Nano-Scale YOLO Detectors in Video Object Tracking

Amr Yasser

May 1, 2026

Abstract

This report presents an empirical analysis comparing the inference speed and tracking efficiency of three nano-scale object detection models: YOLOv10n, YOLOv11n, and YOLOv12n. ByteTrack was utilized as the standard association algorithm across all tests to isolate the pure detection cost. The experiment tracks vehicles in a 30 FPS high-definition (720p) video sequence. Results indicate that while newer architectures like YOLOv12n offer advanced feature extraction capabilities, they exhibit a measurable increase in inference latency compared to the YOLOv10n baseline. We provide a detailed breakdown of predict versus track latency and discuss the practical implications of variance and tail-latency (p95) in production environments.

1 Introduction

Real-time object tracking is a critical component of intelligent transportation systems, surveillance, and autonomous driving. The YOLO (You Only Look Once) family of models has dominated the real-time detection paradigm. With the introduction of YOLOv10n, YOLOv11n, and YOLOv12n, there is a continuous evolution in the trade-off between mean Average Precision (mAP) and computational cost. This experiment benchmarks these three nano-scale models, maintaining a constant tracking algorithm (ByteTrack), precision (FP16), and image resolution (640px) to provide an apples-to-apples comparison of their computational footprint.

2 Experimental Setup

The controlled experiment was executed within a reproducible Kaggle environment. To ensure measurement integrity, 30 warmup frames were discarded before timing commenced, mitigating initial CUDA kernel launch overheads. Synchronization barriers (`torch.cuda.synchronize()`) were strictly enforced.

- **Dataset:** `Test.mp4` (Resolution: 1280x720, 30 FPS, 942 frames, ≈ 31.4 s)
- **Hardware:** Tesla T4 GPU, Torch 2.10.0+cu128
- **Configuration:** `imgsz=640, half=True, conf=0.25, iou=0.45`
- **Target Classes:** Vehicles (COCO classes 2, 3, 5, 7)
- **Tracker:** ByteTrack (`bytetrack.yaml`)

3 Performance Evaluation

3.1 Headline Metrics

The raw performance metrics demonstrate a progressive increase in parameter count and computational complexity from v10n to v12n. The pure detection pass (*Predict*) isolates the network’s forward pass, while the *Track* pass includes detection, ByteTrack association, Kalman filter updates, and bounding box rendering.

Table 1: Aggregate Speed and Detection Metrics

Model	Params	Pred. FPS	Track FPS	Pred. Mean	Track Mean	Track p95	Tracks
YOLOv10n	2,775,520	85.1	64.2	11.78 ms	16.16 ms	17.17 ms	200
YOLOv11n	2,624,080	78.3	60.6	12.81 ms	16.55 ms	17.99 ms	184
YOLOv12n	2,603,056	55.3	47.6	18.10 ms	21.09 ms	22.87 ms	149

3.2 Relative to Baseline

When normalized against YOLOv10n, the computational cost of the newer iterations becomes apparent.

Table 2: Latency Deviation relative to YOLOv10n

Model	Predict ms diff	Track ms diff
YOLOv10n	+0.0%	+0.0%
YOLOv11n	+8.7%	+2.4%
YOLOv12n	+53.6%	+30.5%

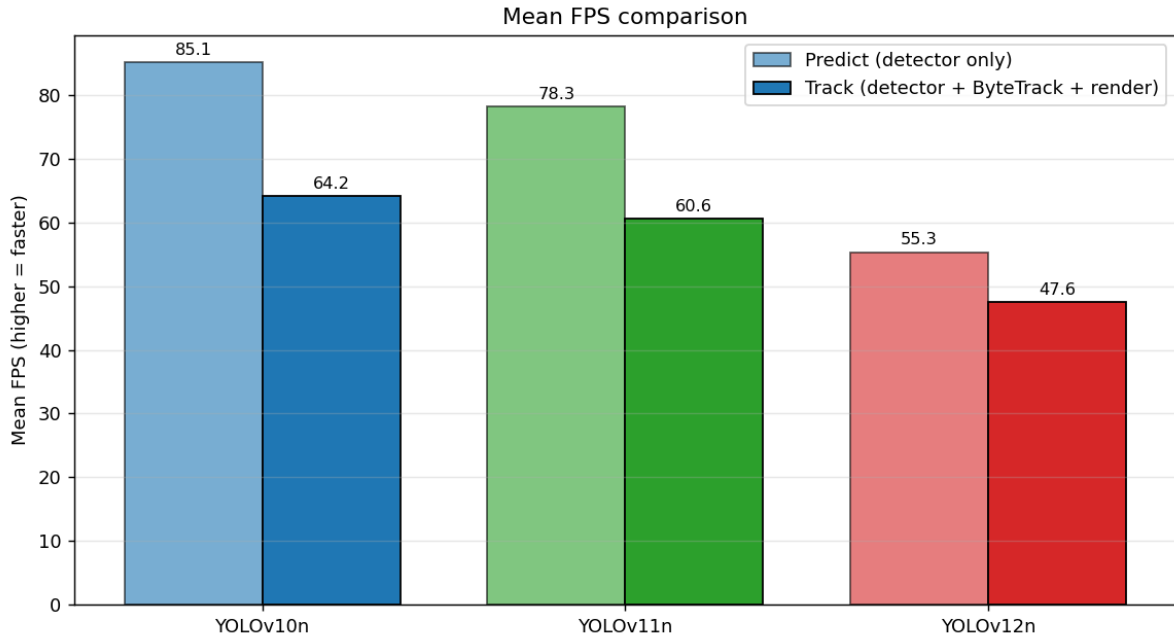


Figure 1: Mean FPS comparison between pure detection and tracking.

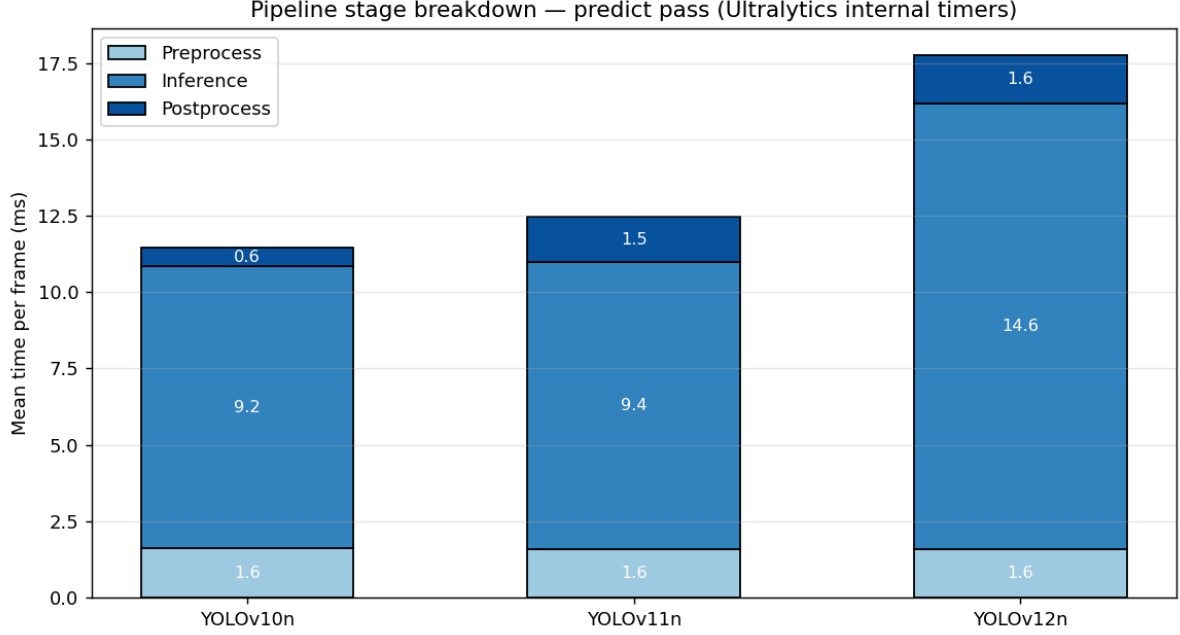


Figure 2: Pipeline stage breakdown isolating the forward pass inference time.

4 Analysis and Discussion

Pure Detector Cost vs Tracker Overhead The *predict ms* metric represents the cleanest signal of architectural cost. The tracking pass adds a relatively constant overhead across all models (roughly 4-5 ms per frame), encompassing ByteTrack logic and OpenCV rendering.

The Importance of Variance Mean metrics can obscure system jitter. The 95th percentile (p95) tracking latency for YOLOv10n is exceptionally tight (17.17 ms vs 16.16 ms mean), indicating high deterministic reliability, which is crucial for strict real-time pipelines. YOLOv12n exhibits higher variance, rendering it more susceptible to frame drops in edge-deployment scenarios.

Detection Density Constraints The number of unique tracked identities varies (200 for v10n vs 149 for v12n). While this may indicate differences in false-positive suppression or identity-switching behavior, sharp divergences in detection density can marginally skew the FPS measurements, as fewer active tracks reduce Kalman filter update costs.



Figure 3: Sample frames with tracking annotations from YOLOv10n, YOLOv11n, and YOLOv12n.

5 Conclusion

For constrained environments requiring maximum throughput, YOLOv10n remains the optimal choice, delivering the highest FPS and lowest latency variance. YOLOv11n offers a competitive middle ground, trading a marginal 8.7% increase in pure detection latency for potentially refined architectural features. YOLOv12n, despite its lower parameter count, incurs a significant 53.6% detection latency penalty, demanding careful consideration if deployed in latency-critical real-time tracking systems.